

Sales Visit Optimizer

GeoAnalytics and Planning Dashboard

Hackathon September 2026

Alessandro Savo, 2021372

Leonardo Santucci, 2282707

Academic Year 2025/2026



SAPIENZA
UNIVERSITÀ DI ROMA



Business Problem

- **Context:** The project helps a federation of three companies plan visits for their sales representatives.
- **Data:** We start from an Excel file containing customers, delivery points, sales agents, and revenue data.
- **The Challenge:** Sales representatives have limited time. The problem is deciding **which customers to visit**, in **which order**, and on **which days**.
- **The Goal:** Cover as much relevant revenue as possible within the available working time.
- **What-if Analysis:** The user can easily compare different scenarios by changing the number of available days and the duration of each visit.



Main User Stories (1/2)

- **Upload & Reuse**

"As a Business Analyst, I want to upload an Excel file, so that I can work with the latest data extracted from the ERP."

- **Company Selection**

"As a Sales Manager, I want to select a company, so that I can analyze only the customers and revenue related to that company."

- **Customer Map**

"As a Sales Manager, I want to see the customers of the selected company on a map, so that I can understand their geographic distribution."



Main User Stories (2/2)

- **Start Date & Working Days**

"As a Sales Manager, I want to choose the starting date of the visit campaign, so that I can plan visits for a specific commercial period."

- **Suggested Visit Sequence**

"As a Sales Manager, I want the system to create a suggested sequence of customer visits, so that I can organize the working day."

- **Recoverable Revenue**

"As a Sales Manager, I want to see the estimated recoverable revenue of the proposed plan, so that I can understand the economic value."

- **What-if Analysis**

"As a Sales Manager, I want to change the number of available days and run the analysis again, so that I can compare different scenarios."



Non-Functional Requirements

- **Usability**

The application should be easy to use and understandable without specific technical knowledge.

- **Reusability**

The application should be reusable with future ERP files having the same logical structure.

- **Portability**

The application should be deployable on another computer using Docker and Docker Compose.

- **Accuracy & Robustness**

Provide clear error messages for invalid files and use real geographic coordinates for customer locations.



From Requirements to Interface

These mockups were used to define the structure of the dashboard before implementation.

Mockup 1

Upload and Scenario Setup

Sales Visit Optimizer

ERP Excel File

Company
WINES ▼

Campaign Start Date

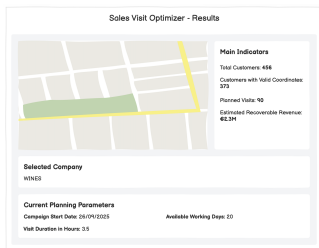
Available Working Days

Visit Duration in Hours

Run Analysis

Mockup 2

Results, Map and KPIs



Mockup 3

Suggested Visit Plan

Suggested Visit Plan

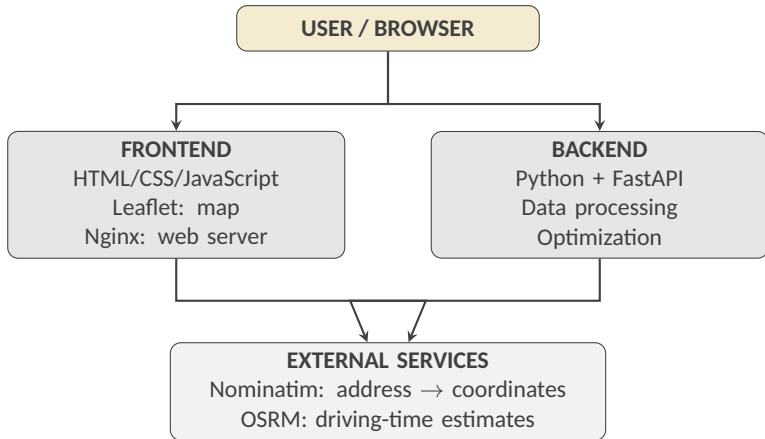
Selected Company
WINES

Visit Agenda

DATE	DAY	TIME	CUSTOMER	CITY	ADDRESS	ESTIMATED REVENUE
01/10/2023	Wednesday	09:00 - 11:00	Customer A	Rome	Via Salaria 56	€1,000
01/10/2023	Wednesday	11:30-13:30	Customer B	Rome	Via Bove 34	€7,500
02/10/2023	Thursday	14:30-16:30	Customer C	Guidonia	Via Roma 144	€3,000
03/10/2023	Friday	9:00-11:00	Customer D	Rome	Via Tiburtina 630	€4,100



Software Architecture





Docker Deployment

Docker

Packages the application and its dependencies.

Docker Compose

Starts the two application services together.

Frontend

Port 8080

Backend

Port 8000

docker-compose.yml

```
backend:
  build: ./backend
  ports: ["8000:8000"]

frontend:
  build: ./frontend
  ports: ["8080:80"]
  depends_on: [backend]
```



Data Processing

The same physical location can appear several times in the Excel file because it can have revenue from different companies.

Example

Customer A - Via Roma 10 - WINES - EUR 1,000

Customer A - Via Roma 10 - VISCONTI - EUR 500

↓ GROUP BY PHYSICAL LOCATION ↓

Customer A - Via Roma 10 - Total relevant revenue

1. Clean the Excel data.
2. Group rows referring to the same physical location.
3. Convert addresses into geographic coordinates.
4. Estimate driving times between locations.



Optimization Problem

INPUTS

- Customers
- Revenue
- Location
- Available days
- Visit duration
- Starting point

CONSTRAINTS

- Working days
- Working hours
- Holidays
- Lunch break
- Visit time
- Driving time

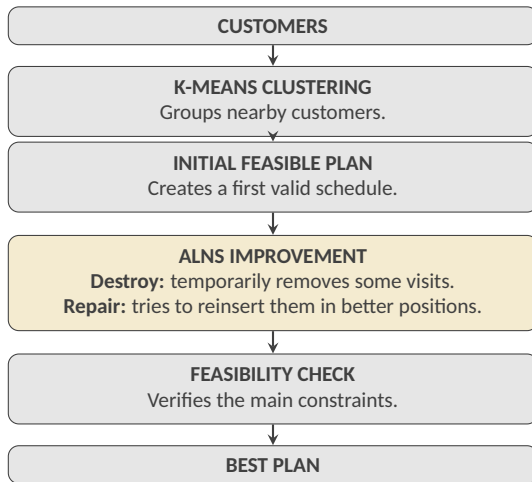
GOAL

Find a feasible visit plan that covers as much revenue as possible.

Testing every possible route would take too long, so we use a heuristic algorithm.



How the Optimizer Works





Simplified Algorithm

Simplified logic — not the actual source code

```
best = initial_plan

for each iteration:
    partial = destroy(current_plan)
    candidate = repair(partial)

    if candidate is feasible:
        current_plan = candidate

        if candidate is better than best:
            best = candidate

return best
```

DESTROY

Remove some visits.

REPAIR

Try to insert them again in better positions.

FEASIBLE

All time and calendar constraints are respected.

BEST

Keep the best plan found so far.



Real Optimizer Code - Destroy

alns.py

```
num_remove = max(1, int(len(
    route.client_indices) *
    random.uniform(0.1, 0.2)))

indices_to_remove = random.sample(
    range(len(route.client_indices)),
    num_remove)

for idx in sorted(indices_to_remove,
                  reverse=True):
    client_idx = \
        route.client_indices.pop(idx)
```

1. Decide how many visits to remove.
2. Randomly choose the visits.
3. Remove them from the current route.

Why?

Removing visits gives the algorithm space to rebuild part of the route differently.



Real Optimizer Code - Repair

alns.py

```
for client_idx in candidates:
    # ... skips if already scheduled ...

    insertion = _find_best_insertion(...)

    if insertion:
        d_idx, pos, new_route, _ = insertion
        solution.day_routes[d_idx] = new_route
        solution.scheduled_client_ids.add(...)
```

Removed customer



Try possible positions

- Day 1 - Position 1
Not feasible
- Day 1 - Position 2
Feasible
- Day 2 - Position 1
Feasible and better



Choose the best feasible position

_find_best_insertion searches for the best valid day and position.



Testing

Test Execution

```
$ python3 -m pytest -q source/test
.....
6 passed in 0.08s
```

Working-time limits

Checks that visits do not exceed the available working day.

Calendar rules

Checks that weekends and national holidays are skipped.

Travel / transfer rules

Checks that travel-related constraints do not create invalid plans.

Final feasibility

Checks again that the final schedule respects the main constraints before it is returned.

Tests verify that the planner respects the main constraints.



Conclusions

WHAT WE LEARNED

- Real business data need cleaning before optimization.
- Geographic accuracy strongly affects route quality.
- Route planning must balance revenue, visit time and driving time.
- Heuristic methods can find useful solutions without testing every possible route.
- Docker makes the application easier to reproduce and deploy.



Live Demo

Now let's see the Sales Visit Optimizer in action.